Audit Chat — rate-limited conversational interface over the finding store

SPEC-SW-012 v0.1 DRAFT Jeremie Nunez

1 Context

The audit chat (internally referred to as `sweep chat` in the codebase) exposes a per-entity conversational surface over the finding store. A reviewer selects an entity, asks a question, and gets a streamed answer backed by the entity's audited data and any previously extracted findings. Because every turn incurs an LLM call with web search enabled, and because the same reviewer tends to retype variants of the same question, the service must enforce per-user rate limits, deduplicate repeat questions, and window the finding history so the prompt stays within budget. This spec defines those three guardrails and their observable behaviour.

2 Goals

- Enforce a per-user request rate limit on the chat endpoint; return a typed `rate_limited` event when the cap is exceeded.
- Deduplicate semantically equivalent repeated questions inside a short window and serve the previous answer instead of calling the model again.
- Window the finding history injected into the system prompt: bounded by count, ordered most-recent-first.
- Guarantee message persistence ordering: the user turn is stored before the model is called; the assistant turn is stored after the stream completes.
- Fail open on repository errors: a streaming client always receives a terminal `done` event, even after an internal error.

3 Non-Goals

Non-Goal

This spec does not cover: (a) the content of the system prompt or quality of model output, (b) long-term chat archival, analytics, or export, (c) per-organisation or tiered rate limits beyond a single per-user counter, (d) the schema of extracted findings (covered elsewhere), (e) multi-user collaborative chat on the same entity.

4 Invariants

Invariant

Rate limiting is per-user and time-bounded: at most `RATE_LIMIT` chat requests per rolling `RATE_WINDOW_SECS` window. A request beyond the cap is refused before any model call is dispatched.

Invariant

Deduplication is scoped to a single (`userId`, `entityId`) pair: a normalized user message that matches a previous message within the last `DEDUP_WINDOW_SECS` returns the cached prior assistant response, no new model call, and no rate-limit decrement beyond the accounting of the duplicate.

Invariant

Finding history injected into the prompt is bounded: at most `MAX_FINDINGS_IN_PROMPT` findings, ordered by creation time descending. Older findings are omitted, never summarized.

Invariant

Chat message persistence is ordered around the model call: the user message is persisted *before* the stream is opened; the assistant message is persisted *after* the stream terminates normally. A stream that errors mid-flight persists no assistant message.

Invariant

Every invocation of `chat` yields exactly one terminal event of type `done` or `error`, even if the underlying repository or model call raises. No client is left hanging.

5 Interface

Public surface consumed by the transport layer.

```
export interface ChatMessage {
  role: "user" | "assistant";
  content: string;
  timestamp: string;        // ISO 8601
}

export interface Finding {
  id: string;
  entityId: string;
  category: string;
  title: string;
  summary: string;
  confidence: number;       // 0..1
  createdAt: string;
}

export type ChatEvent =
  | { type: "delta"; data: { text: string } }
  | { type: "finding"; data: Finding }
  | { type: "rate_limited"; data: { retryAfterSecs: number } }
  | { type: "duplicate"; data: { servedFromCache: true } }
  | { type: "error"; data: { error: string } }
  | { type: "done"; data: Record<string, never> };

export interface AuditChatRepository {
  checkRateLimit(userId: string): Promise<
    { allowed: true } | { allowed: false; retryAfterSecs: number }
  >;
  findDuplicate(
    userId: string,
```

```
    entityId: string,
    normalizedMessage: string,
  ): Promise<ChatMessage | null>;
  appendMessage(entityId: string, userId: string, msg: ChatMessage):
     Promise<void>;
  getHistory(entityId: string, userId: string): Promise<ChatMessage[]>;
  getFindings(entityId: string, limit?: number): Promise<Finding[]>;
}

export interface AuditChatService {
  chat(
    entityId: string,
    userId: string,
    message: string,
  ): AsyncGenerator<ChatEvent>;
}
```

6 Scenarios

Scenario: Nominal turn produces a streamed answer and stores both messages

Given user `U` has made no recent chat requests
And entity `E` has $k <$ `MAX_FINDINGS_IN_PROMPT` previous findings
When `chat(E, U, "what changed last week?")` is iterated to completion
Then the generator yields ≥ 1 `delta` event, then a `done` event
And `appendMessage` is called exactly twice: once with role `user` before the stream, once with role `assistant` after

Scenario: Rate limit exceeded

Given user `U` has already made `RATE_LIMIT` successful chat requests in the current window
When `chat(E, U, ...)` is invoked
Then the generator yields a single `rate_limited` event with a positive `retryAfterSecs`
And then yields `done`
And no model call is issued
And no message is appended to history

Scenario: Duplicate question is served from cache

Given user `U` asked `"How is pricing evolving?"` on entity `E` $<$ `DEDUP_WINDOW_SECS` ago
And the assistant answer was stored
When `U` asks `"how is pricing evolving?"` again on `E`
Then the service yields a `duplicate` event
And yields the prior assistant content as one or more `delta` events
And then yields `done`
And no new model call is issued

Scenario: Finding history is windowed

Given entity `E` has 100 stored findings
When a chat turn on `E` builds its system prompt
Then the prompt embeds exactly `MAX_FINDINGS_IN_PROMPT` findings
And those findings are the `MAX_FINDINGS_IN_PROMPT` with the most recent `createdAt`

Scenario: Mid-stream error is reported and terminated

Given the model provider raises partway through streaming
When `chat` is iterated
Then the generator yields zero or more `delta` events, then one `error` event, then a `done` event
And no assistant message is persisted
And the previously-persisted user message is not rolled back

7 Acceptance Criteria

Acceptance Criterion AC-1

Given a user with no recent chat activity, iterating `chat` to completion persists exactly one user message and one assistant message, in that order.

Acceptance Criterion AC-2

Given a user at the rate-limit cap, `chat` yields a `rate_limited` event followed by `done`, with no intermediate `delta` and no persisted message.

Acceptance Criterion AC-3

Given a duplicate user message within `DEDUP_WINDOW_SECS`, `chat` yields a `duplicate` event and replays the cached assistant content without invoking the model.

Acceptance Criterion AC-4

Message normalization for dedup is case-insensitive and whitespace-collapsing: `"Hello World"` and `" hello world "` deduplicate to each other.

Acceptance Criterion AC-5

When `getFindings(entityId)` returns more than `MAX_FINDINGS_IN_PROMPT` results, the prompt builder includes only the `MAX_FINDINGS_IN_PROMPT` most recent.

Acceptance Criterion AC-6

Every terminating path of the generator — nominal, rate-limited, duplicate, mid-stream error, repository error — ends with exactly one `done` event.

Acceptance Criterion AC-7

When the model stream throws mid-flight, the assistant message is *not* persisted; the user message persisted before the stream remains.

Acceptance Criterion AC-8

The rate-limit counter is incremented on successful request admission and is not incremented on a `duplicate` hit beyond its own accounting.

8 Traceability

AC	Test file	Test name
AC-1	`packages/sweep/tests/unit/audit-chat.spec.ts`	`persists user then assistant on n`
AC-2	`packages/sweep/tests/unit/audit-chat.spec.ts`	`rate-limit short-circuits the str`
AC-3	`packages/sweep/tests/unit/audit-chat.spec.ts`	`duplicate replays cached answer`
AC-4	`packages/sweep/tests/unit/dedup-normalize.spec.ts`	`normalization is case/whitespace`
AC-5	`packages/sweep/tests/unit/audit-chat-prompt.spec.ts`	`finding window is bounded and mos`
AC-6	`packages/sweep/tests/integration/audit-chat.spec.ts`	`every path yields a single done e`
AC-7	`packages/sweep/tests/integration/audit-chat.spec.ts`	`mid-stream error does not persist`
AC-8	`packages/sweep/tests/unit/rate-limit.spec.ts`	`counter semantics across admissio`

9 Open Questions

- Should rate limits be tiered (e.g. higher cap for admin roles), or is a single per-user cap sufficient for the interview scope?
- Is a pure string-equality dedup enough, or do we want embedding-based near-duplicate detection? Current draft is string-normalization only.
- Should duplicate hits still be stored in history (with an origin marker), or skipped entirely?
- What is the right default for `MAX_FINDINGS_IN_PROMPT` (current proposal: 20)?